

Machine-Assisted Error Discovery in Conversational AI Systems

Maeda F. Hanafi
maeda.hanafi@ibm.com
IBM Research AI
San Jose, CA, USA

Robert J. Moore
rjmoore@us.ibm.com
IBM Research AI
San Jose, CA, USA

Frederick Reiss
frees@us.ibm.com
IBM Research AI
San Jose, CA, USA

David Wood
dawood@us.ibm.com
IBM Research AI
Yorktown Heights, NY, USA

Yannis Katsis
yannis.katsis@ibm.com
IBM Research AI
San Jose, CA, USA

Mohammad H. Falakmasir
mohammad.falakmasir@ibm.com
IBM watsonx Orders
San Jose, CA, USA

Changchang Liu
changchang.liu33@ibm.com
IBM Research AI
Yorktown Heights, NY, USA

ABSTRACT

Troubles in speaking, hearing, and understanding occur routinely in any kind of conversational setting. The natural flow of conversation includes methods for “repairing” such troubles by repeating or paraphrasing all or parts of prior turns. In the case of conversational AI systems, these troubles occur due to failure of different components of the system such as the speech recognition, natural language understanding, and natural language generation. Such errors may occur infrequently, but still often enough to have a significant impact on key performance indicators (KPIs). Identifying the root cause of these errors is a complex task that requires a team to meticulously examine and interpret the interaction between the voice agent and customers. In this work, we present an interactive system, DTTool, that surfaces system-generated annotations that hint at anomalous events that lead to candidate errors that impact KPIs and demonstrate how the team could discover unknown errors using DTTool.

CCS CONCEPTS

• **Human-centered computing** → **Interactive systems and tools**; • **Applied computing** → *Business intelligence; Enterprise applications.*

ACM Reference Format:

Maeda F. Hanafi, Frederick Reiss, Yannis Katsis, Robert J. Moore, David Wood, Mohammad H. Falakmasir, and Changchang Liu. 2024. Machine-Assisted Error Discovery in Conversational AI Systems. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3613905.3651120>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CHI EA '24, May 11–16, 2024, Honolulu, HI, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0331-7/24/05
<https://doi.org/10.1145/3613905.3651120>

1 INTRODUCTION

IBM watsonx Orders is an AI voice agent that automates order-taking for the quick-service restaurant (QSR) industry. The IBM watsonx Orders agent has been deployed in production for over three years at a major U.S. quick-service restaurant chain and currently serves over 200 drive-through lanes. The system interacts with customers by engaging them in conversation in the same way that a human crew member would while taking their order.

The conversational AI system is capable of complex interactions such as customizing a menu item that has multiple components and optional extras; or informing customers that the item they have requested is unavailable and suggesting a substitute. These capabilities rely on a multi-modal AI model that not only listens to the customer’s speech, but also interacts with sensors and other systems like the restaurant’s point of sale device.

The agent is under constant development, and improved versions of the agent roll out on a biweekly basis. Advising the development team on what improvements to make is a task of the *Data Triage team* at IBM watsonx Orders. The Data Triage team has access to the full set of telemetry and voice recordings from millions of ordering sessions as well as data on the *key performance indicators* (KPIs) that are important to IBM watsonx Orders clients. These metrics include *automation rate* (the fraction of orders that are handled entirely by the AI), *escalation rate* (the fraction of orders that ended up being handled by a human crew member), *dialog efficiency* (the rate at which the AI can take orders, adjusted for order complexity), and customer satisfaction. For every potential improvement, the team needs to answer the question: *What would be the impact on our key performance indicators if the system were to be improved in this way?*

Consider the example session data in Figure 1. In this case, what happened is that, in line 06, the customer actually said, “I’ll have a cone please”, but the AI’s speech recognition model transcribed that utterance as “i’ll have a code please”. The AI interprets the mis-transcribed phrase as a request to enter a code from the restaurant’s mobile app and asks, “What’s your code?”, leaving the customer confused. Variations on this particular exchange occurred 437 times

Excerpt of a session transcript

```

      :
      :
05 A: What would you like to order?
06 H: i'll have a code please
07 A: What's your code?
08 H: no code
      :
      :

```

Partial application log of the session

Event Timestamp	Event Type	Event
2023-05-29T01:30:12	TTS	What would you like to order?
2023-05-29T01:30:15	ASR	[ay,l,hh,ae,v,ah,k,ow,d,p,l,iy,z]
2023-05-29T01:30:15	ASR	i'll have a code please
2023-05-29T01:30:15	Paraphrase	I have a code [CodeCommand]
2023-05-29T01:30:16	Canvas	[Empty Canvas]
2023-05-29T01:30:17	TTS	What's your code?
2023-05-29T01:30:30	ASR	no code
2023-05-29T01:30:30	Paraphrase	I don't have a code. [NoCodeCommand]
2023-05-29T01:30:31	Canvas	[Empty Canvas]

Figure 1: Left: Example of a transcript of a session between human (H) and voice agent (A). Right: Application log of the same session. Reading only the transcript and application log makes it hard to discern the trouble occurring in the conversation, but based on the audio, the human wanted a cone and the agent misunderstood it as code.

within the period of one week in a production deployment resulting in 136 escalations to a human crew member to complete the order.

To understand how the Data Triage team connects this raw data to their KPIs, we conducted interviews with data scientists from the team and observed them in several hands-on working sessions. Our hands-on sessions also allowed us to study how the members of the Data Triage team analyze the system logs and audio data to understand in detail what happened during a particular order-taking session.

At a high level, the process the Data Triage team takes to analyze session logs from the conversational AI system breaks down into four major steps:

- (1) **Examine the data.** Search for patterns of failure by reviewing a representative sample of escalated orders.
- (2) **Formulate a hypothesis about what's going on.** Identify patterns of similar, suspicious events across sessions and the KPI(s) that are affected by this pattern.
- (3) **Annotate and mine suspicious events.** Use data mining tools to retrieve instances of suspicious patterns. Define these patterns precisely enough that automated tools can mine the patterns from system logs.
- (4) **Estimate impact.** Compare the aggregate values of KPI(s) in sessions where the error pattern is present with sessions where the error pattern is not present.

This process requires domain experts to spend a great deal of time examining complex data and writing code to further mine the data for the raw statistics that feed the remaining steps.

Our general solution for improving the efficiency of this process was to identify and assign parts of the workflow that a human can do well and parts where a machine does well. The machine first annotates and mines suspicious events (Step 3) from all sessions and performs an impact analysis (Step 4) that filters suspicious events so only errors that correlate with changes to KPIs are kept. We surface these suspicious events as system-generated annotations through a UI where they were presented in conjunction with the session logs. The UI's design, inspired by methods of Conversation Analysis [23, 30, 32], presents the session logs in an intuitive introspective manner accompanied by system-generated annotations that hint at candidate errors on suspicious events. The analyst can

then examine (Step 1) the sessions and suspicious events in the context of the session log and formulate a hypothesis about what went wrong (Step 2). By reorganizing the error discovery workflow, we can enable human and machine to more effectively collaborate on error finding and discovery.

Our contributions are as follows: (1) a novel machine-assisted framework for discovering errors and their impact to KPIs using system-generated *annotations*, (2) DTTool which has its design informed by concepts from Conversation Analysis and guided by insights gained from analyzing the Data Triage team's workflow for discovering hard-to-find errors in IBM watsonx Orders, and (3) qualitative evaluations of the DTTool with the Data Triage team where errors impacting the KPIs were discovered "in the wild".

2 RELATED WORK

A large body of work is dedicated to developing, building, and designing chatbots and conversational AI systems [5, 7, 11, 17, 19, 21, 24, 25, 36]. There are also works focusing on specific components of a conversational AI system, such as the text-to-speech (TTS) [10], natural language understanding (NLU) [26], and automatic speech recognition (ASR) [9, 34] components. However, these works focus on the *development* of a conversational AI system instead of issues that occur during its lifetime, i.e., its *maintenance*, which include discovering new errors, measuring their impact on KPIs, and ensuring that system performance does not degrade with respect to KPIs.

Substantial work has also been done on designing dialog mechanisms for conversational AI system, which includes handling misunderstandings, providing explanations, listing options, and asking clarifying questions [4, 14]. Many conversational AI systems are designed with these dialog mechanisms, as outlined in the field of Conversation Analysis [23, 30, 32]. However, they do not deal with the production-related aspects of a conversational AI system in an enterprise setting, where discovering errors and identifying error instances that impact KPIs are important.

There is existing work on error detection in conversational AI systems; however, such work focuses on detecting and correcting errors on the fly during production and adjusting the agent's response accordingly [6, 28, 35]. Even with these mechanisms in

place, there are errors after the fact that still need to be spotted, especially ones with significant impact on KPIs. Our work focuses on discovering errors in historical session logs, where the impact of individual errors is subtle, but their aggregate impact across sessions is significant. Existing models for automatically detecting errors or performing error correction during production are important but are not able to discover the class of hard-to-find errors identified by the Data Triage team. There are also tools for analyzing logs [2]. DTTool differs from log viewers and query tools as it focuses on helping analysts explore log session data between humans and agent.

In our work, we abstract the transcript between the voice agent of the conversational AI system and the human, and similar to [38], we model the interaction as event sequences. The Data Triage team essentially performs a computer-assisted pattern matching of event sequences across thousands of sessions to identify the patterns that point to an error in the system. There are several works on mining patterns from event sequences [8]. Frequence [27] is one such tool that takes on a visual and interactive approach to event sequence mining. Frequence utilizes frequent subgraph mining (FSM) to handle event sequences that contain complex, hierarchical information. While we considered the FSM approach for our work, the stochastic nature of how errors manifest and propagate in event sequences of transcripts is usually not in the form of a repeated subgraph. This was also observed in a similar context when using FSM over graphs of data provenance to find errors propagating in large data processing pipelines since many errors do not manifest structurally [12].

There have also been natural language processing-based approaches for improving the performance of dialog systems [18, 26]. In one such work, outliers that are either errors or unique instances are automatically detected and used as part of their data quality strategy [18]. In another similar line of work, deep learning models identify troubles in production session logs and label such instances with a better semantic interpretation [26]. Both approaches are primarily used as strategies to improve the data for future iterations of supervised training of conversational AI systems.

In our interviews and collaboration with the Data Triage team, we observed that their process of error discovery in a conversational AI system requires human judgment and analysis on a large statistical scale involving several thousands of session logs. We redesigned their workflow to have humans and machines effectively collaborate in discovering errors from application logs of a conversational AI system. We take inspiration from similar tooling in the context of debugging, which includes debugging software [1, 16], debugging data pipelines [13, 40], and debugging machine learning/AI systems [3, 33, 39].

3 DTTOOL

We employed User-Centered Design (UCD) [37], where we involved the Data Triage team in an iterative design process focusing on their needs. The responsibilities of the Data Triage team involve identifying instances where the voice agent or conversation deviates from the expected path, classifying them as errors, and prioritizing their resolution. We developed a technique for flagging such deviations in an automated fashion by comparing two versions of the voice

agent’s paraphrased interpretation of the customer’s intent: (1) the intent extracted from the application logs and (2) an alternate version extracted from intensive offline analysis (see Appendix A.1 for more details). The *differences* capture how the outputs differ between the production model and the offline model:

- Replace, e.g. **Replace ‘code’ with ‘cone’**: To get from the production model’s output, “I want a code”, to the offline model’s output, “I want a cone”, replace ‘code’ with ‘cone’.
- Insert, e.g. **Insert ‘code’**: To get from the production model’s output, “I want a code”, to the offline model’s output, “I want a code code”, insert ‘code’ into the output of the production model.
- Delete, e.g. **Delete ‘I’**: To get from the production model’s output, “I want a code”, to the offline model’s output, “want a code”, delete ‘I’ from the output of the production model.

DTTool consists of a data model and backend services to facilitate the offline analysis. The offline analysis consists of the tasks a machine can do well, specifically where it analyzes all the sessions and identifies differences that correlate with changes in KPIs. DTTool then surfaces the differences as annotations through the UI, as shown in DTTool’s architecture in Figure 2. The pages of the UI and the primary tasks they support are described in Table 1.

3.1 Inspecting a Session with the Analyst Page

The Analyst Page (Figure 3) displays information an analyst needs to inspect a single session and identify errors within the session and their root causes. Suppose an analyst takes interest in a particular system-generated annotation that has hinted at the root cause of an error in a particular session, e.g. the annotation containing the difference **Replace ‘code’ with ‘cone’**. To find other sessions exhibiting the error, the analyst clicks on the search icon inside the annotation, and the Similar Snippets Panel populates with snippets of customer utterances from other sessions exhibiting the same difference, where each utterance may potentially exhibit the error (Figure 3).

Due to how different errors manifest differently (e.g. through the difference in intent or through phonetic features, etc.), the analyst may also find similar snippets phonetically by hovering over a particular customer utterance, which then triggers the search functionality to appear, labeled as “Find similar snippets”. Clicking on that search function populates the Similar Snippets Panel with customer utterances from other sessions that are phonetically similar to the customer utterance. Phonetic similarity search identifies a set of human utterances that closely resemble the selected utterance in terms of phonetics, indicating a similarity in their audio content [15, 20]. To achieve this, we utilize the 1-gram histogram of phonemes extracted from human utterances and assess phonetic similarity using the *Euclidean* distance metric. Searching based on audio features is much less cumbersome than experimenting with possible mistranscriptions of the text by typing it in the search input fields.

Given that the Data Triage team primarily uses Python for analytics, we also created an external Python library that communicates with DTTool’s REST API (see Figure 2). Sessions and events displayed in the search pages and the Similar Snippets Panel can then

Task	Page in UI	Page Description
T1: Given an instance of an error in a session, find similar instances of that error in other sessions.	Analyst Page	Visualizes a session log transcript between a human customer and a voice agent as a conversation, a design that is inspired by Conversation Analysis [22, 29, 31] (see Figure 3). Also displays similar customer utterances from other sessions that are similar (1) by phonetic features or (2) by the difference in the system-generated annotation.
T2: Discover novel and hard-to-find errors in the sessions.	Summary Page	Displays a list of system-generated annotations ranked by their prevalence in the session log dataset for a particular period (Figure 4).
	All Sessions Page & All Annotations Page	Provides basic search for sessions based on session metadata and annotation metadata (Figure 5 in the Appendix). Search results are persisted in the UI's cache and can be navigated by the “Previous” and “Next” buttons in the top-left corner of the Analyst Page (Figure 3).

Table 1: Primary tasks supported by DTTool

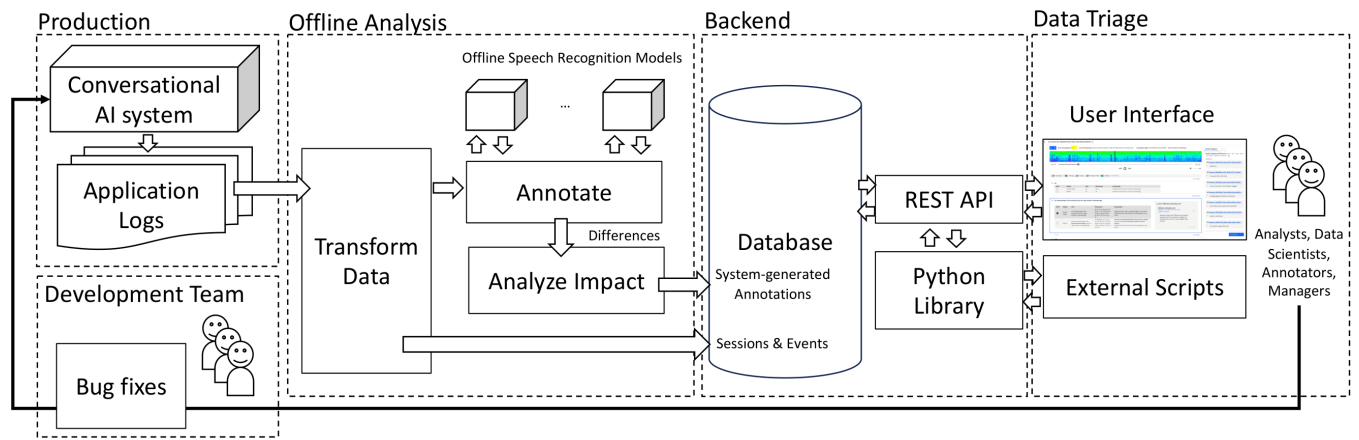


Figure 2: Overall architecture of our human-in-the-loop solution, DTTool, which consists of the offline analysis, backend components, and UI. The offline analysis (Section A.1) transforms the raw application into our generic, internal data model (Section A.2) and then generates annotations of differences the production model has from other offline speech recognition models. The resulting sessions, events, and annotations are stored in the backend database. They are surfaced to stakeholders of the Data Triage team through the UI or external scripts (using our Python library that communicates with the REST API). A report of errors is then sent to the development team to inform them which bugs/errors to fix in the production conversational AI system.

be loaded into the Python script for further analysis. Or a list of suspicious sessions from an external script can be loaded into the UI and further investigated with the Analyst Page.

Using the Analyst Page, an analyst can easily follow the conversation as it unfolded in real-time. The spectrogram visualization enables the analyst to see at a glance where human speech occurs. The horizontal bands clearly distinguish speech from silence, as well as from other types of sounds. Moreover, the bands display changes in pitch, such as rising intonation at the end of an utterance, which typically invites the other to speak next, as well as different voices, overlapping talk, and more. Other visual representations of sound, such as waveforms, do not provide as rich information. The analyst can also toggle the layers in the transcript panel to do a quick scan of the transcript or a more in-depth investigation of the root cause of an error in the transcript. Additional details of the Analyst Page are described in the Appendix, Section A.3.

3.2 Discovering Errors with the Summary Page

The Summary Page (Figure 4) has 3 panels with the primary goal of helping an analyst discover errors (T2). An analyst begins by inspecting the ranked list of the differences of system-generated annotations shown on the top-left panel. Clicking on a difference triggers a preview of it where (1) the right panel lists the system-generated annotations with those differences and (2) the bottom panel is populated with additional statistics about the selected difference. If the analyst decides to further investigate the system-generated annotations on the right-hand side, he may group the annotations by session or annotation metadata, such as store location, annotation type, etc. This grouping can help the analyst discover patterns across annotations for a particular difference, e.g. the difference **Delete 'coke'** occurs when serving the day and breakfast menu but not the night menu. The analyst may want to investigate if that combination of menu item and time of day is

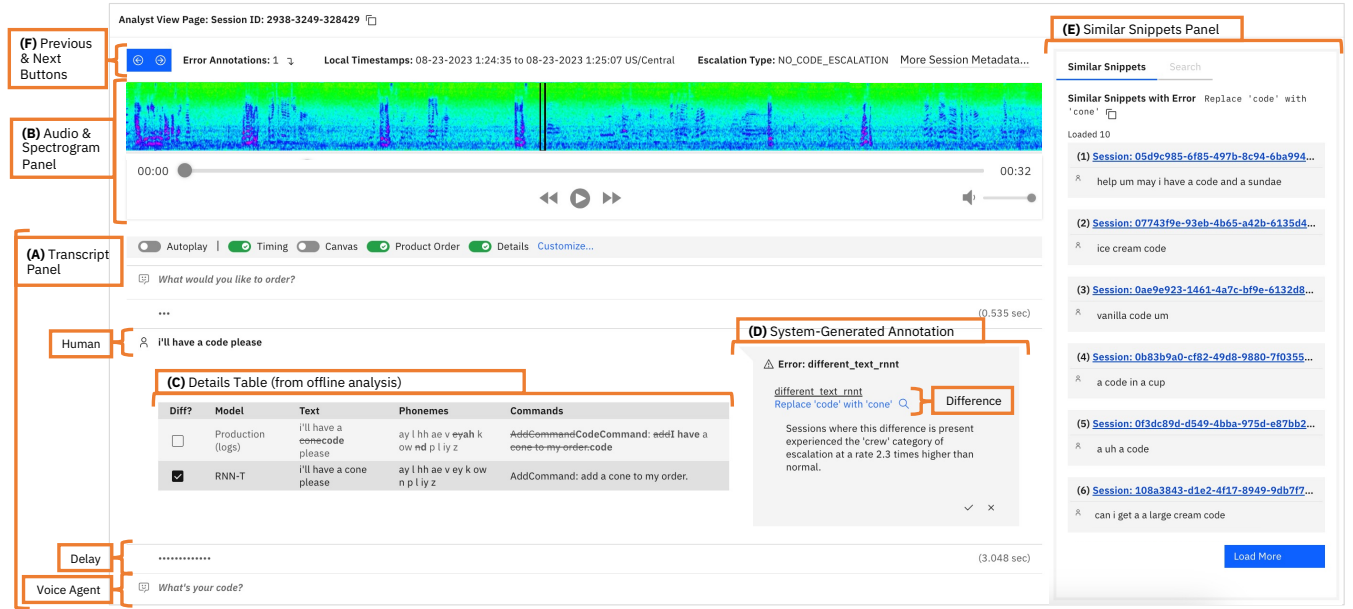


Figure 3: The Analyst Page displays a session log between a voice agent and a human. The transcript panel (A) displays the transcript from Figure 1. It syncs with the spectrogram as the audio player runs (B). The transcript panel has knobs to toggle additional layers, such as delays between the events. The “Details” toggle reveals the Details Table showing outputs of the offline analysis (C). Given the human customer utterance, the Details Table shows the outputs of the model at production and models ran during offline analysis. There is a difference between the texts of the two models, where the word “code” was replaced with “cone” by the offline model, the RNN-T model. We denote this difference as **Replace ‘code’ with ‘cone’, as shown in the system-generated annotation (D). These annotations appear at the lines of suspicious events correlating to candidate errors where (1) a difference between offline model and production model outputs occurred and (2) the difference is correlated with a high escalation rate (or any KPI). The Similar Snippets Panel (E) lists customer utterances from other sessions that are similar to a user-selected utterance in the current session.**

correlated with an error. In the bottom-left panel, to mitigate information overload, we only exhibit column statistics that we observed the Data Triage team found valuable for identifying errors.

4 EVALUATION

4.1 Qualitative User Study with the Data Triage team

We ran a qualitative user study with 7 members of the Data Triage team. We gave them a hands-on tutorial on how to use DTTool (which at the time only had the search page and the Analyst Page implemented). We showed them how to search for sessions using the All Sessions Page, read the session logs using the Analyst Page, populate the Similar Snippets panel, and navigate between the previous and next sessions of a search result from the Analyst Page. We then gave them 30 minutes to complete the task of finding at least 5 sessions that exhibit the *same error*¹ (T1). Finally, they filled out a survey of their experience using the DTTool.

The Analyst Page, which was designed using aspects of Conversation Analysis, received most of the positive feedback. Participants

in particular appreciated the functionality of finding similar snippets. (“This sort of search [the Similar Snippet search] is something we would do. We’d have to like, go to a Python notebook, create the logic and then we could select extra sessions; that saves ton of time by just having this in a UI.”).

Another aspect that multiple participants commented on was the system-generated error annotations. While some participants found them promising, they did not like the fact that they were only shown in the context of individual sessions on the Analyst Page. This design, rooted in a *bottom-up approach* to error analysis, initiates the analysis from individual sessions, aiming to identify general error patterns. In contrast to this approach, some users preferred a more guided *top-down approach*, where the error analysis process could start instead from a list of system-generated annotations sorted based on their severity or impact (“What about a top-down situation where I would like to say: Hey, I wanna see the most common error and go figure out what it is?”). This observation made us rethink the error analysis workflow, leading to implementing the Summary Page that enables a top-down error discovery approach. One analyst comments after using the Summary Page, “If you do it manually without the tool, you can trace it back to a transcription error but it’s not obvious at first. But with the tool you can start with

¹An error in one session is said to be the same as another error from another session if it goes away with the same fix.

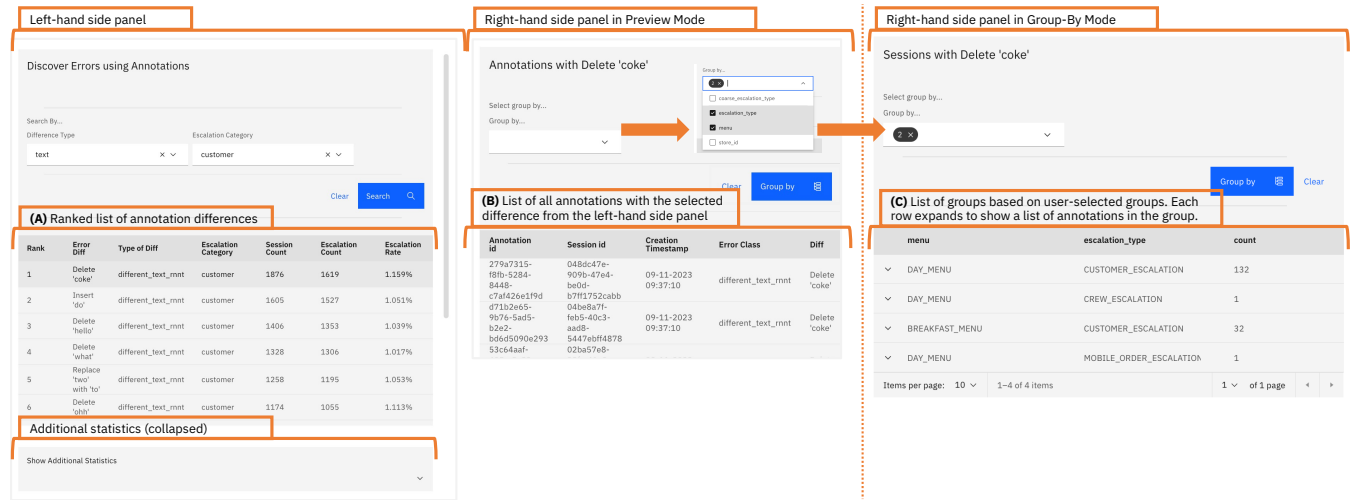


Figure 4: The Summary Page displays a ranked list of differences (A) on the left-hand side panel and allows a user to filter the list based on escalation category, e.g. “customer”, and difference type (“Type of Diff” column), e.g. filtering by “text” list differences that exist between the production model’s text outputs and the offline model’s text outputs which indicated with “different_text_rnnnt” values. The user selects a difference to inspect annotations that appear on the right-hand side panel. The annotations are listed (B) or displayed in a group-by table according to user-defined groupings (C).

these [annotations] and see if they [annotations] point to any other trouble.”

After this qualitative user study and implementation of features based on the feedback, DTTool was put into production for the Data Triage team to use. Since then, a total of 8 new issues were discovered and prioritized for resolution. The following section describes how the Data Triage team found some of these errors.

4.2 Discovering Errors “In The Wild”

In this section, we present unknown errors that were discovered (T2) by members of the Data Triage team using DTTool. Here were the steps that Alice², a data scientist from the Data Triage team, took to discover three different errors of varying impact on the KPI, which at the time of writing was escalation rate. These steps demonstrate how Alice was able to discover issues by utilizing the system-generated annotations that hinted at suspicious events, leading to the creation of tickets for the development team to resolve.

Alice begins by writing a Python script to load a list of differences, exported from the REST API of DTTool, ranking them according to escalation rate. Note that this ranked list of differences is also available on the Summary Page of DTTool, allowing non-technical users similar to Alice to discover errors. Alice inspects the list and decides to look into **Replace ‘add’ with ‘update’**, wherein the production model predicted that the customer intent was to add a new item while the offline model predicted an update for an existing item. Alice switches over to the UI, navigates to the All Sessions Page, and searches for **Replace ‘add’ with ‘update’** by typing in the search field, where DTTool automatically suggests it as one of

the options. Our UI then returns all the sessions annotated with this difference. Alice then clicks on one of the sessions, directing to the Analyst Page, and immediately navigates to the highlighted section with the difference and validates the system annotation by listening to the audio. The production model failed to transcribe a customer utterance “not a double” but the offline model accurately identified the intent through the correct transcription. Alice also notices that the comment in the system-generated annotation says, “Sessions where this difference is present experience 2.6 times higher than the normal escalation rate in the ‘crew’ category along with 4.1 times higher rates in the ‘heuristic’³ category and 2.9 times higher rates in the ‘customer’ escalation category.” This is an important aspect, as the decision for whether to further pursue a particular error can easily be made based on the system-generated annotation and in turn the KPI at hand, which is to minimize the escalation rate and prioritize issues that result in higher escalation rates. Alice then decides to use the All Sessions Page to retrieve sessions that are escalated by the ‘crew’ and contain **Replace ‘add’ with ‘update’**. After reviewing multiple sessions, Alice identifies multiple variations of requests for updates not currently supported by the IBM watsonx Orders and creates a ticket for the development team.

Alice examines the system-generated ranked list of differences, which was generated using a Python script she wrote (as described in Section 4.2). This time Alice examines **Replace ‘one’ with ‘want’**, which means the production model outputs ‘one’ but the offline model outputs ‘want’ for the same word. She notes that this could potentially result in a wrong quantity for a particular menu item

²Names and proprietary information were modified for protecting privacy, but the errors found and the steps taken to find them are accurate.

³IBM watsonx Orders has a watchdog facility that monitors model inference and looks for patterns such as lack of progress after a fixed amount of time or failures of internal checks during model inference. The ‘heuristic’ escalation category contains sessions where this watchdog triggered an escalation.

and then takes similar steps as the previous difference to verify the error. The particular sessions she reviews, however, do not result in an escalation to a human crew member, as the agent ended up adding the correct items to the canvas (even though the agent transcribed ‘want’ as ‘one’). As she listens to the audio of more sessions and examines the details table of the utterances annotated with the difference, she recognizes that the difference is mostly present when the customer has an accent or when there is a noisy background, both of which are issues the team is currently working on.

Alice then looks at another difference, this time at the paraphrase level. She takes interest in the difference Insert ‘wrong’, indicating the offline model understood that the customer was telling the agent that there was something wrong with the order, while the production model did not. Alice performs the same set of steps as before, searching for sessions with the difference and navigating on the Analyst Page. She also clicks the error annotation in the Analyst Page to populate the Similar Snippets Panel with Insert ‘wrong’. Alice comes across a session where the customer says, “chicken nuggets i’m sorry chicken patty”, but the agent misunderstood and added a chicken nugget to the order. This is a new error that was not seen before and Alice immediately creates a ticket for this error.

5 CONCLUSION & FUTURE WORK

In this paper, we presented a case study on the Data Triage team and a human-in-the-loop solution for error discovery in conversational AI systems. We observed the Data Triage team’s workflow, which involved a combination of large statistical scale analysis and manually sifting through thousands of session log data. We reordered this process so that some of the work can be done by the machine, particularly the part where a machine performs an offline analysis of the audio and identifies suspicious events that correlate with the escalation rate. The human then inspects the sessions annotated by the machine as opposed to random samples of sessions. We showed the effectiveness of this tool with qualitative evaluations of how the Data Triage team used the tool to discover unknown errors.

There are several promising directions for future research and development. One of the directions involves implementing and experimenting with a more advanced version of search, such as incorporating phonetic similarity in conjunction with event sequences which is a type of search the Data Triage team expressed interest in. Another important direction involves enhancing the overall pipeline for detecting suspicious events during offline analysis. While differences in model outputs currently serve as effective indicators of potential errors, as our production model advances and matches the performance of the offline model, it becomes crucial to consider alternative error-detection features. These features could include delays between events, such as delays between customer and voice agent interactions, or repair patterns that occur on the conversational level. Finally, we plan to expand the scope of the offline analysis to refine its filters for features that align with KPIs. This adaptation is necessary to accommodate development of the conversational AI system and evolving business priorities.

REFERENCES

- [1] Azza Abouzied, Joseph Hellerstein, and Avi Silberschatz. 2012. DataPlay: Interactive Tweak and Example-Driven Correction of Graphical Database Queries. In *Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology* (Cambridge, Massachusetts, USA) (UIST '12). Association for Computing Machinery, New York, NY, USA, 207–218. <https://doi.org/10.1145/2380116.2380144>
- [2] S. Alspaugh, Beidi Chen, Jessica Lin, Archana Ganapathi, Marti Hearst, and Randy Katz. 2014. Analyzing Log Analysis: An Empirical Study of User Log Mining. In *28th Large Installation System Administration Conference (LISA14)*. USENIX Association, Seattle, WA, 62–77. <https://www.usenix.org/conference/lisa14/conference-program/presentation/alspaugh>
- [3] Saleema Amershi, Max Chickering, Steven M. Drucker, Bongshin Lee, Patrice Simard, and Jina Suh. 2015. ModelTracker: Redesigning Performance Analysis Tools for Machine Learning. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) (CHI '15). Association for Computing Machinery, New York, NY, USA, 337–346. <https://doi.org/10.1145/2702123.2702509>
- [4] Zahra Ashktorab, Mohit Jain, Q. Vera Liao, and Justin D. Weisz. 2019. Resilient Chatbots: Repair Strategy Preferences for Conversational Breakdowns. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3290605.3300484>
- [5] Tom Bocklisch, Joe Faulkner, Nick Pawlowski, and Alan Nichol. 2017. Rasa: Open Source Language Understanding and Dialogue Management. *ArXiv* abs/1712.05181 (2017). <https://api.semanticscholar.org/CorpusID:19971625>
- [6] I. Bulyko, K. Kirchhoff, M. Ostendorf, and J. Goldberg. 2005. Error-correction detection and response generation in a spoken dialogue system. *Speech Communication* 45, 3 (2005), 271–288. <https://doi.org/10.1016/j.specom.2004.09.009>
- [7] Mikhail Burtsev, Alexander Seliverstov, Rafael Airapetyan, Mikhail Arkhipov, Dilyara Baymurzina, Nickolay Bushkov, Olga Gureenkova, Taras Khakhulin, Yuri Kuratov, Denis Kuznetsov, Alexey Litinsky, Varvara Logacheva, Alexey Lymar, Valentin Malykh, Maxim Petrov, Vadim Polulyakh, Leonid Pugachev, Alexey Sorokin, Maria Vikhreva, and Marat Zaynutdinov. 2018. DeepPavlov: Open-Source Library for Dialogue Systems. In *Proceedings of ACL 2018, System Demonstrations*. Association for Computational Linguistics, Melbourne, Australia, 122–127. <https://doi.org/10.18653/v1/P18-4021>
- [8] Mohammed Elseidy, Ehab Abdelhamid, Spiros Skiadopoulos, and Panos Kalnis. 2014. GraMi: Frequent Subgraph and Pattern Mining in a Single Large Graph. *Proc. VLDB Endow.* 7, 7 (mar 2014), 517–528. <https://doi.org/10.14778/2732286.2732289>
- [9] Manaal Faruqi and Dilek Hakkani-Tür. 2022. Revisiting the Boundary between ASR and NLU in the Age of Conversational Dialog Systems. *Computational Linguistics* 48, 1 (04 2022), 221–232. https://doi.org/10.1162/coli_a_00430 arXiv:https://direct.mit.edu/coli/article-pdf/48/1/221/2006612/coli_a_00430.pdf
- [10] Haohan Guo, Shaofei Zhang, Frank K. Soong, Lei He, and Lei Xie. 2021. Conversational End-to-End TTS for Voice Agents. In *2021 IEEE Spoken Language Technology Workshop (SLT)*, 403–409. <https://doi.org/10.1109/SLT48900.2021.9383460>
- [11] Xu Han, Michelle Zhou, Matthew J. Turner, and Tom Yeh. 2021. Designing Effective Interview Chatbots: Automatic Chatbot Profiling and Design Suggestion Generation for Chatbot Debugging. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (CHI '21). Association for Computing Machinery, New York, NY, USA, Article 389, 15 pages. <https://doi.org/10.1145/3411764.3445569>
- [12] Maeda F Hanafi. 2020. *Human-in-the-loop Tools for Constructing and Debugging Data Extraction Pipelines*. Ph.D. Dissertation. New York University Tandon School of Engineering.
- [13] Maeda F Hanafi, Azza Abouzied, Marina Danilevsky, and Yunyao Li. 2020. WhyFlow: Explaining Errors in Data Flows Interactively.. In *DaSH@ KDD*.
- [14] Mohit Jain, Pratyush Kumar, Ramachandra Kota, and Shwetak N. Patel. 2018. Evaluating and Informing the Design of Chatbots. In *Proceedings of the 2018 Designing Interactive Systems Conference* (Hong Kong, China) (DIS '18). Association for Computing Machinery, New York, NY, USA, 895–906. <https://doi.org/10.1145/3196709.3196735>
- [15] Shigeto Kawahara. 2021. Phonetic bases of sound symbolism: a review. *Preprint*. *PsyArXiv*. <https://doi.org/10.31234/2021>
- [16] Amy J. Ko and Brad A. Myers. 2004. Designing the Whyline: A Debugging Interface for Asking Questions about Program Behavior. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vienna, Austria) (CHI '04). Association for Computing Machinery, New York, NY, USA, 151–158. <https://doi.org/10.1145/985692.985712>
- [17] Olexsii Kuchaiev, Jason Li, Huyen Nguyen, Olexsii Hrinchuk, Ryan Leary, Boris Ginsburg, Samuel Krizan, Stanislav Beliaev, Vitaly Lavrukhin, Jack Cook, Patrice Castonguay, Mariya Popova, Jocelyn Huang, and Jonathan M. Cohen. 2019. NeMo: a toolkit for building AI applications using Neural Modules. *CoRR* abs/1909.09577 (2019). arXiv:1909.09577 <http://arxiv.org/abs/1909.09577>

- [18] Stefan Larson, Anish Mahendran, Andrew Lee, Jonathan K. Kummerfeld, Parker Hill, Michael A. Laurenzano, Johann Hauswald, Lingjia Tang, and Jason Mars. 2019. Outlier Detection for Improved Data Quality and Diversity in Dialog Systems. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 517–527. <https://doi.org/10.18653/v1/N19-1051>
- [19] Sungjin Lee, Qi Zhu, Ryuichi Takanobu, Zheng Zhang, Yaoqin Zhang, Xiang Li, Jinchao Li, Baolin Peng, Xiujuan Li, Minlie Huang, and Jianfeng Gao. 2019. ConvLab: Multi-Domain End-to-End Dialog System Platform. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, Florence, Italy, 64–69. <https://doi.org/10.18653/v1/P19-3011>
- [20] Jeff Mielke. 2012. A phonetically based metric of sound similarity. *Lingua* 122, 2 (2012), 145–163.
- [21] Alexander Miller, Will Feng, Dhruv Batra, Antoine Bordes, Adam Fisch, Jiasen Lu, Devi Parikh, and Jason Weston. 2017. ParlAI: A Dialog Research Software Platform. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Copenhagen, Denmark, 79–84. <https://doi.org/10.18653/v1/D17-2014>
- [22] Robert J. Moore, Sungeun An, and Guang-Jie Ren. 2023. The IBM natural conversation framework: a new paradigm for conversational UX design. *Human-Computer Interaction* 38, 3-4 (2023), 168–193. <https://doi.org/10.1080/07370024.2022.2081571> arXiv:<https://doi.org/10.1080/07370024.2022.2081571>
- [23] Robert J Moore and Raphael Arar. 2019. *Conversational UX design: A practitioner's guide to the natural conversation framework*. Morgan & Claypool.
- [24] Tim Paek and Roberto Pieraccini. 2008. Automating spoken dialogue management design using machine learning: An industry perspective. *Speech Communication* 50, 8 (2008), 716–729. <https://doi.org/10.1016/j.specom.2008.03.010> Evaluating new methods and models for advanced speech-based interactive systems.
- [25] Alexandros Papangelis, Mahdi Namazifar, Chandra Khatri, Yi-Chia Wang, Piero Molino, and Gökhan Tür. 2020. Plato Dialogue System: A Flexible Conversational AI Research Platform. *CoRR* abs/2001.06463 (2020). arXiv:2001.06463 <https://arxiv.org/abs/2001.06463>
- [26] Sunghyun Park, Han Li, Ameen Patel, Sidharth Mudgal, Sungjin Lee, Young-Bum Kim, Spyros Matsoukas, and Ruhi Sarikaya. 2021. A Scalable Framework for Learning From Implicit User Feedback to Improve Natural Language Understanding in Large-Scale Conversational AI Systems. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 6054–6063. <https://doi.org/10.18653/v1/2021.emnlp-main.489>
- [27] Adam Perer and Fei Wang. 2014. Frequence: Interactive Mining and Visualization of Temporal Frequent Event Sequences. In *Proceedings of the 19th International Conference on Intelligent User Interfaces (Haifa, Israel) (IUI '14)*. Association for Computing Machinery, New York, NY, USA, 153–162. <https://doi.org/10.1145/2557500.2557508>
- [28] Plamen Prodanov and Andrzej Drygajlo. 2005. Bayesian networks based multi-modality fusion for error handling in human-robot dialogues under noisy conditions. *Speech Communication* 45, 3 (2005), 231–248. <https://doi.org/10.1016/j.specom.2004.10.015> Special Issue on Error Handling in Spoken Dialogue Systems.
- [29] Harvey Sacks, Emanuel A. Schegloff, and Gail Jefferson. 1974. A Simplest Systematics for the Organization of Turn-Taking for Conversation. *Language* 50, 4 (1974), 696–735. <http://www.jstor.org/stable/412243>
- [30] Emanuel A Schegloff. 1992. Repair after next turn: The last structurally provided defense of intersubjectivity in conversation. *American journal of sociology* 97, 5 (1992), 1295–1345.
- [31] Emanuel A. Schegloff. 2007. *Sequence Organization in Interaction: A Primer in Conversation Analysis*. Vol. 1. Cambridge University Press. <https://doi.org/10.1017/CBO9780511791208>
- [32] Emanuel A Schegloff, Gail Jefferson, and Harvey Sacks. 1977. The preference for self-correction in the organization of repair in conversation. *Language* 53, 2 (1977), 361–382.
- [33] Prithviraj Sen, Yunyao Li, Eser Kandogan, Yiwei Yang, and Walter Lasecki. 2019. HEIDL: Learning Linguistic Expressions with Deep Learning and Human-in-the-Loop. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, Florence, Italy, 135–140. <https://doi.org/10.18653/v1/P19-3023>
- [34] Yik-Cheung Tam, Yun Lei, Jing Zheng, and Wen Wang. 2014. ASR error detection using recurrent neural network language model and complementary ASR. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2312–2316. <https://doi.org/10.1109/ICASSP.2014.6854012>
- [35] F. Torres, L.F. Hurtado, F. García, E. Sanchis, and E. Segarra. 2005. Error handling in a stochastic dialog system through confidence measures. *Speech Communication* 45, 3 (2005), 211–229. <https://doi.org/10.1016/j.specom.2004.10.014> Special Issue on Error Handling in Spoken Dialogue Systems.
- [36] Stefan Ultes, Lina M. Rojas-Barahona, Pei-Hao Su, David Vandyke, Dongho Kim, Iñigo Casanueva, Paweł Budzianowski, Nikola Mrkšić, Tsung-Hsien Wen, Milica Gašić, and Steve Young. 2017. PyDial: A Multi-domain Statistical Dialogue System Toolkit. In *Proceedings of ACL 2017, System Demonstrations*. Association for Computational Linguistics, Vancouver, Canada, 73–78. <https://aclanthology.org/P17-4013>
- [37] Karel Vredenburg, Ji-Ye Mao, Paul W. Smith, and Tom Carey. 2002. A survey of user-centered design practice. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Minneapolis, Minnesota, USA) (CHI '02). Association for Computing Machinery, New York, NY, USA, 471–478. <https://doi.org/10.1145/503376.503460>
- [38] Kam Kwai Wong, Xingbo Wang, Yong Wang, Jianben He, Rong Zhang, and Huamin Qu. 2024. Anchorage: Visual Analysis of Satisfaction in Customer Service Videos Via Anchor Events. *IEEE Transactions on Visualization and Computer Graphics* (2024), 1–13. <https://doi.org/10.1109/tvcg.2023.3245609>
- [39] Yiwei Yang, Eser Kandogan, Yunyao Li, Prithviraj Sen, and Walter S. Lasecki. 2019. A Study on Interaction in Human-in-the-Loop Machine Learning for Text Analytics. In *IUI Workshops*. <https://api.semanticscholar.org/CorpusID:77392827>
- [40] Ce Zhang, Christopher Ré, Michael Cafarella, Christopher De Sa, Alex Ratner, Jaeho Shin, Feiran Wang, and Sen Wu. 2017. DeepDive: Declarative Knowledge Base Construction. *Commun. ACM* 60, 5 (apr 2017), 93–102. <https://doi.org/10.1145/3060586>

A APPENDIX

A.1 Offline Analysis

We automated the steps of the Data Triage team’s workflow (Section 1), that are amenable to automation, and we adjusted the overall workflow so that these steps happen in an offline analysis process before data enters the tool (Figure 2). The offline analysis ingests the raw application logs produced in production from the conversational AI system and produces annotations hinting at suspicious events that potentially point to errors in the conversational AI system. There are several steps in the offline analysis, as shown in Figure 2: (1) transforming the data (Section A.1.1), (2) annotating suspicious events between offline model outputs and the production model output using *differences* (Section A.1.2), and (3) filtering the annotations based on their impact on KPIs (Section A.1.3) and loading the data into the backend database.

A.1.1 Transforming the Raw Session Logs. The IBM watsonx Orders automated order-taking (AOT) system runs locally at all locations. A daemon on each edge node asynchronously uploads audio recordings and detailed application logs to cloud storage. These raw application logs serve as the input to our offline processing. We currently use a 500,000-session sample of the much larger IBM watsonx Orders data set. This size is large enough for the analyses we describe in the following sections to produce useful results, while being small enough to allow us to iterate quickly while building our offline analysis scripts. The logs contain a complete history of all internal application events, as well as all messages that passed between IBM watsonx Orders and other components that IBM watsonx Orders interacts with while taking orders. These other components include the point-of-sale device, the audio system, and the detectors for approaching automobiles. After parsing, we reconstruct a detailed session transcript, including not only what the customer and agent said during the session, but also the sequence of internal events and messages that were triggered by each customer utterance.

A.1.2 Annotating Suspicious Events with Differences. The session transcript contains the sequence of turns in each session as experienced by the deployed system. When the system makes a mistake, it is often because the production automatic speech recognition (ASR) model misheard what the customer said. An analyst can identify these problems in speech recognition by listening to the session’s audio recording. That process is very time-consuming, so we added a re-transcription phase to provide information about what sessions may be worth listening to.

The IBM watsonx Orders’s ASR model needs to produce results with low latency on edge hardware. These real-time constraints lead to some reduction in accuracy. It is possible to achieve higher accuracy in an offline setting with a model that uses more computation and has access to the entire session’s audio.

We first re-transcribe the audio of the entire session using a special ASR model that is tuned to provide highly accurate information about when each word of customer speech occurred. This special ASR model is tuned to produce accurate transcriptions at the expense of resources not available at production time, i.e. high computational cost, low timing accuracy, etc. This additional, offline phase of ASR processing produces a session transcription with

fewer errors than the transcript we extracted from the production logs. We use this secondary transcript to look for cases where the production ASR model made mistakes that led to escalations.

The presence of a mistake in the outputs of the production model is not necessarily a cause for concern. It is normal for ASR models to make mistakes, especially when they are processing audio from a drive-through lane next to a busy highway. However, most mistakes do not cause problems, because language and conversation are redundant. More importantly, an IBM watsonx Orders in production has multiple layers of protection that allow the system to recover from nearly all ASR errors without crew member intervention. Moreover, not all differences between our two models’ outputs are due to the model with lower average accuracy being wrong. Sometimes our high-accuracy model makes a mistake, and sometimes both models make different mistakes at the same time. For all of these reasons, simply surfacing the differences in the output of the two ASR models would not help focus the attention of human experts.

Producing a useful set of “suspicious” differences in model output requires several steps of additional model inference and statistical analysis. The goal of these processing steps is to isolate errors in speech recognition that occur frequently and that also cause the system to misbehave in a way that its existing fail-safes cannot repair.

Rather than focusing only on differences in the raw transcribed text, we run this text through further processing and look at downstream results for a total of three types of differences, shown as columns in the Details Table in Figure 3: (1) different text, (2) different phonemes in the phonetic transcription of that text, and (3) different *paraphrase commands*, which represents the understood intent after running the phonetic transcription through the IBM watsonx Orders’s intent detection model. The upstream differences, i.e. phonetic and intent differences, often remain constant in cases when differences in the transcribed text are more random and variable.

A.1.3 Filtering Annotations that Impact KPIs. From these base sets of differences, we start by filtering out differences that do not occur in at least 100 sessions per week. We rank the remaining differences by examining and comparing the aggregate escalation status of sessions that contain an instance of the difference with that of sessions that do not contain the difference. The AOT system in production tracks 20 different ways that the system may escalate a session to a human crew member. We group these categories into the broader categories of customer, crew, and system-initiated escalations. Only differences whose presence is highly correlated with an increase in one of these categories are loaded into our database for display in the user interface. The threshold we currently use requires an increase of at least a factor of two at a significance level of 99%.

A.2 Internal, Generic Data Model

After the offline analysis, the raw application data has been transformed into our internal, generic data model and is then loaded into the database, as depicted in Figure 2. The data model is schema-agnostic and could be employed in other conversational AI systems. The data model defines three objects: sessions, events, and annotations. A **session** collection contains metadata about a session and

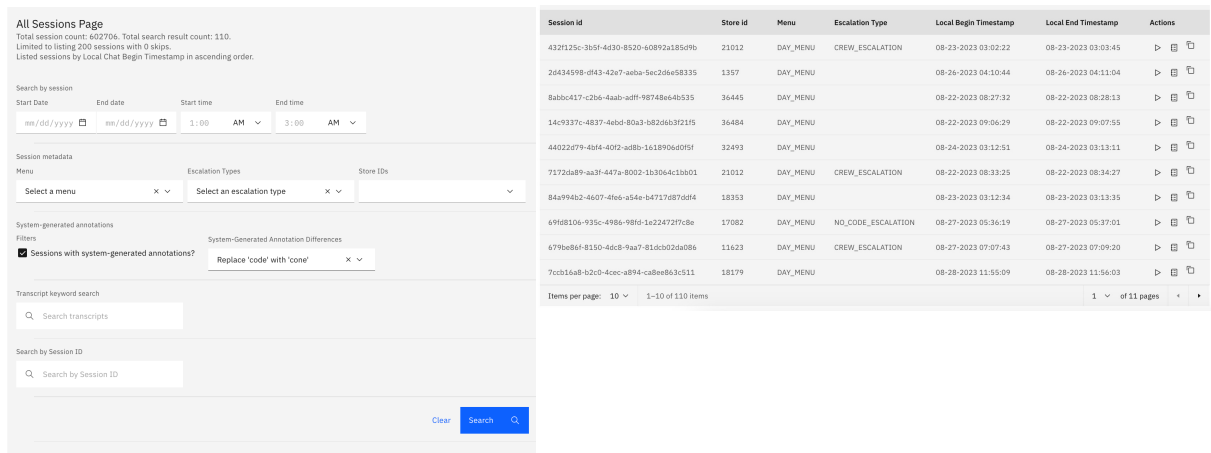


Figure 5: The All Sessions Page offers basic search functionality for finding sessions based on session and annotation metadata. The left screenshot shows a search for sessions containing system-generated annotations with the difference **Replace ‘code’ with ‘cone’ and the right screenshot displays the resulting sessions.**

is associated with a session ID. A session consists of an ordered set of **events**, containing events such as human utterances and the voice agent’s responses. A session essentially forms a directed acyclic graph of event nodes ordered by time and causality. An **annotation** collection stores any analysis or information after the fact of the session and events occurring in production, including system-generated annotations and human-written annotations. An annotation references an event or a session object.

A.3 Additional UI Details

A.3.1 Details in the Analyst Page. In Figure 3, the Analyst Page consists of toggles that reveal layers in the transcript panel. By default, the transcript always shows the voice agent’s lines, the human customer utterances, any system-generated annotations associated with each human utterance, and the conversational AI system’s response to the human customer utterances, e.g. “Product

Order”: product order update. The analyst may toggle to any one of the layers: (1) “Timing”: time delays between each event, (2) “Canvas”: detailed product order update, (3) “Details”: details on the outputs of the offline analysis. The details toggle displays the internal conversational system’s response to each human utterance, an ASR event, which the conversational system then translates to a paraphrase command (the human-readable version of what the system understood from the human’s utterance).

A.3.2 Searching with the All Sessions Page and All Annotations Page. In addition to the global view of sessions and annotations provided by the Summary Page, the All Sessions Page and All Annotations Page provide basic search functionality based on session and annotation metadata. Figure 5 shows the All Sessions Page. The All Annotations Page is very similar and lists annotations that match the search criteria instead of sessions like in the All Sessions Page.